QUIT: A Human-in-the-Loop Platform for AI Research Automation

Xinchen Han [1]

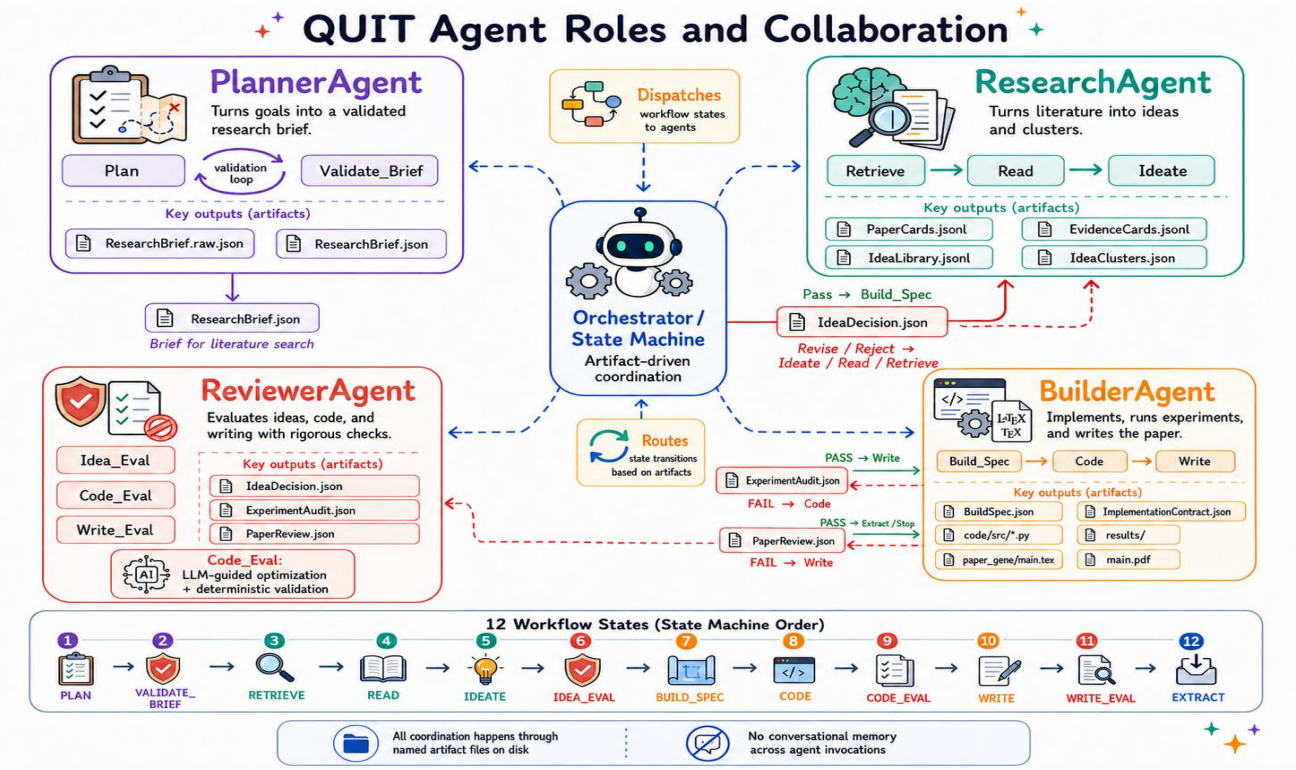

Figure 1. QUIT agent roles and collaboration. See Appendix. A for more details.

Abstract

Large language models (LLMs) can help researchers search the literature, generate ideas, write code, and draft papers. However, in current practice, these capabilities are often used through fragmented interactions hidden inside long chat histories, making the resulting research process difficult to reproduce, audit, or resume after failure. We present **QUIT**, a human-in-the-loop platform for AI research automation that organizes the research process into four stages: **Q**uery, **U**nderstand, **I**mplement, and **T**ell. Rather than using conversational memory as the primary interface between stages, QUIT externalizes intermediate decisions and outputs into structured artifacts. This design avoids the token cost and contextual instability of repeatedly carrying long interaction histories, while making the research process easier to trace, inspect, and reproduce. Moreover, QUIT supports stage-level invocation and repair, allowing researchers to locally inspect intermediate artifacts, identify failure points, and intervene with timely corrections.

Code: QUIT

1. Introduction

LLMs (Singh et al., 2025; DeepSeek-AI, 2026; Anthropic, 2026) are increasingly used across the research process. Researchers ask LLMs to search for related work, summarize papers, propose algorithmic variants, write experimental code, analyze results, and draft manuscripts. These capabilities make LLMs attractive not only as writing assistants, but also as components of AI research automation systems. However, most current uses still take the form of fragmented

[1]Institut Polytechnique de Paris, France. Correspondence to: Xinchen Han <xinchen.han@telecom-sudparis.eu>.

The AI4Research Workshop at ICML 2026, Seoul, South Korea.

prompt-response interactions. Intermediate evidence, design decisions, code changes, result files, and failure recovery steps are often buried in long chat histories or scattered across temporary files (Yang et al., 2026; Jiabin Tang, 2025; Zhang, 2026; Liu et al., 2026).

This creates a reproducibility, auditability, and controllability gap. When an LLM-assisted research trajectory produces a promising idea or result, it can be difficult to answer basic questions: Which papers supported the idea? Which assumptions were extracted from the literature? Which method specification was actually implemented? Did the reported metrics come from real experiment outputs? If a code-generation or writing step failed, where should the workflow resume? At the same time, researchers rarely want a fully black-box automation pipeline. They need to inspect evidence before accepting an idea, revise specifications before implementation, check generated results before writing, and intervene when a failure occurs.

We introduce **QUIT**, a human-in-the-loop platform for AI research automation. QUIT abbreviates four stages. **Q**uery retrieves papers, reference repositories, and user-provided local literature as structured source cards. **U**nderstand reads selected papers, extracts evidence cards, clusters evidence, and synthesizes evidence-grounded ideas. **I**mplement converts a selected idea into a BuildSpec contract, generates experiment code, executes the project, repairs failures, and uses LLM-guided code evaluation with deterministic validation to enforce the declared contract. **T**ell drafts and reviews a paper artifact using the BuildSpec, evidence, and actual experiment outputs. Together, these stages turn a researcher-provided topic into a persistent research trajectory that can be inspected, modified, and resumed.

The core design principle of QUIT is that long-horizon research automation should communicate through inspectable artifacts rather than hidden conversational memory. Every major stage writes durable files under a run directory. These artifacts provide stable interfaces between stages: later stages read what earlier stages explicitly produced, rather than reconstructing the research state from a long dialogue. This design reduces the token cost and contextual instability of repeatedly carrying long interaction histories, while making the process easier to trace and reproduce.

To make artifact-driven automation usable in practice, QUIT supports stage-level invocation and a researcher-facing interface. A user can run the full workflow end-to-end, or select a start state and a stop state to execute only part of the workflow. This allows researchers to inspect evidence cards before ideation, revise or validate the BuildSpec before implementation, verify whether reported metrics are present in result files, and rerun failed stages without restarting from the beginning. In this sense, QUIT is not designed as a fully autonomous agent, but as a human-supervised research platform that exposes the research process as a sequence of controllable and reproducible states.

The contributions of this paper are summarized as follows:

- We introduce QUIT, a human-in-the-loop platform for AI research automation organized around four stages: Query, Understand, Implement, and Tell.
- We propose an artifact-driven state-transition mechanism that turns the system from a black-box automation pipeline into an inspectable and resumable research process.
- We demonstrate both end-to-end execution and researcher-controlled partial execution through a state-machine backend and a researcher-facing interface, enabling users to inspect artifacts, diagnose failures, guide reruns, and resume from saved states.

2. The QUIT Workflow

QUIT exposes AI research automation as an artifact-driven state machine, as shown in Fig. 2. Given a researcher-provided topic, domain, objective, and constraints, the system advances through four stages: **Q**uery, **U**nderstand, **I**mplement, and **T**ell. These stages are implemented through twelve public workflow states: `PLAN` ⇒ `VALIDATE_BRIEF` ⇒ `RETRIEVE` ⇒ `READ` ⇒ `IDEATE` ⇒ `IDEA_EVAL` ⇒ `BUILD_SPEC` ⇒ `CODE` ⇒ `CODE_EVAL` ⇒ `WRITE` ⇒ `WRITE_EVAL` ⇒ `EXTRACT`. Each public state has explicit input artifacts, output artifacts, validation rules, and failure transitions; evaluation states may invoke internal review and revision subroutines.

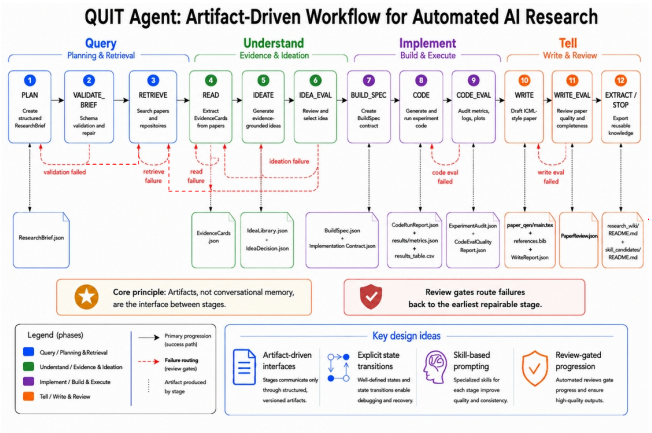

Figure 2. The workflow of QUIT organizes AI research automation into four stages: Query, Understand, Implement, and Tell. By exposing each state and artifact, QUIT allows researchers to inspect, repair, and resume the workflow.

The purpose of this design is not merely to automate a sequence of research tasks, but to make the sequence controllable. A researcher can inspect what each state produced,

identify where a failure occurred, modify intermediate artifacts, and resume execution from a selected state. Unlike a conversational agent that carries the entire interaction history in context, QUIT externalizes intermediate state into durable artifacts such as research briefs, paper cards, evidence cards, idea decisions, BuildSpec contracts, generated code, experiment results, audit reports, paper drafts, and trace files. These artifacts serve as stable interfaces between stages: later stages read structured files produced by earlier stages, rather than reconstructing the research state from long dialogue history.

2.1. Query: Planning and Retrieval

The **Query** stage transforms an initial topic into a structured retrieval plan and evidence source set.

`PLAN` creates a research brief describing the topic, domain, constraints, search & build budget, and expected outputs.

`VALIDATE_BRIEF` checks whether the brief satisfies the schema and repairs common formatting issues when possible. If validation fails, the workflow routes back to `PLAN`, allowing the brief to be regenerated or manually corrected.

After validation, `RETRIEVE` searches for relevant papers, project repositories, and user-provided local literature. The system can query configurable sources, deduplicate candidates, score them by relevance and diversity, optionally download references, and store selected sources as structured paper and repository cards. If retrieval fails, the workflow retries retrieval or returns to planning.

2.2. Understand: Evidence and Ideation

The **Understand** stage converts retrieved literature into structured research knowledge.

`READ` extracts compact evidence cards from selected papers. Each evidence card records the task, method, experimental setting, claims, metrics, limitations, and transferable idea seeds. This produces an explicit evidence base that researchers can inspect before accepting downstream ideas.

`IDEATE` then clusters the evidence and synthesizes candidate research ideas grounded in the extracted literature.

`IDEA_EVAL` reviews these candidates and selects a promising direction for implementation. If the evidence is insufficient or the idea review fails, the workflow can route back to `RETRIEVE`, `READ`, or `IDEATE`. This prevents later stages from building on unsupported ideas and gives researchers a clear intervention point before implementation begins.

2.3. Implement: Build and Execute

The **Implement** stage turns a selected idea into an executable experiment.

`BUILD_SPEC` first creates `BuildSpec.json`, the central contract of the system. The BuildSpec specifies the target task, method, baselines, metrics, logging schema, constraints, success criteria, and paper obligations. It makes the intended experiment explicit before code is generated, giving researchers a concrete artifact to inspect or revise.

`CODE` generates a standalone experiment project from the BuildSpec. The implementation is staged: the system constructs an implementation contract, generates core modules, generates experiment and evaluation modules, normalizes configuration paths, resolves the device, installs missing dependencies when appropriate, executes experiments, and repairs failures.

`CODE_EVAL` combines deterministic validation with an optional LLM-guided performance review and revision loop to check whether the generated experiment actually satisfies the BuildSpec. A successful run is not sufficient by itself: the validation checks that required metrics, baselines, logs, and results are present and consistent with the declared contract. If `CODE_EVAL` fails, the workflow routes back to `CODE` for targeted repair. This review gate reduces the risk that a later paper draft reports metrics or comparisons that were never produced by the experiment.

2.4. Tell: Write and Review

The **Tell** stage turns verified artifacts into a paper-style output.

`WRITE` drafts a manuscript from the BuildSpec, evidence cards, paper cards, and compact summaries of actual result files. The writer is constrained to use available experiment artifacts and should not invent missing values or unsupported claims.

`WRITE_EVAL` reviews the paper artifact. It checks compilation status, citation validity, metric coverage, result consistency, appendix completeness, and whether limitations or missing results are honestly discussed. If the paper review fails, the workflow routes back to `WRITE`.

The terminal `EXTRACT` state exports reusable knowledge, such as research notes or skill candidates, for future runs.

2.5. Stage-Level Control and Failure Routing

A key feature of QUIT is that failures are treated as structured information rather than terminal exceptions. Review gates identify the earliest repairable stage: invalid briefs return to planning, retrieval failures return to planning or retrieval, insufficient evidence can return to retrieval, reading, or ideation, broken code returns to code generation, failed code evaluations return to code repair, and failed paper reviews return to writing.

This failure routing enables stage-level control. Researchers

can invoke a specific state, inspect its artifacts, modify or repair intermediate files, and resume execution from that point. For example, a researcher can stop after `READ` to inspect evidence cards, rerun only `IDEATE` after modifying the evidence set, resume from `BUILD_SPEC` after editing the method specification, or rerun `WRITE` after updating result artifacts.

In this way, QUIT supports human-supervised automation: the system automates repetitive research operations while keeping intermediate decisions, outputs, and failures visible to the researcher.

3. Demonstration and Verification

3.1. Researcher-Facing Interface

To make artifact-driven automation practical for researchers, we provide a researcher-facing interface on top of the state-machine backend. Although all intermediate outputs are stored as files, directly navigating a run directory can be cumbersome when a workflow contains many states, reports, logs, generated source files, result tables, and paper artifacts. The interface exposes the same state-machine abstraction used by the backend: researchers can inspect the current run state, browse generated artifacts, view validation and audit reports, select a start state and a stop state, and launch partial execution without rerunning the entire workflow.

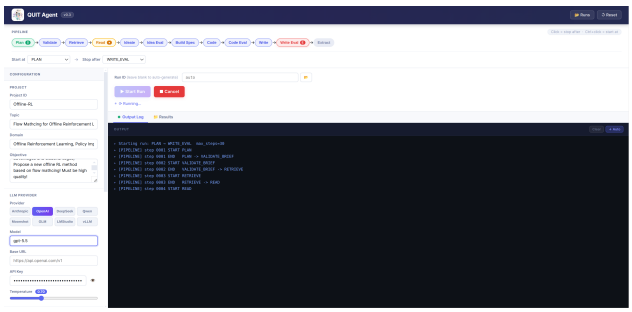

Figure 3. Researcher-facing interface for QUIT.

By exposing intermediate artifacts and state transitions, the interface turns QUIT into an interactive human-in-the-loop research system. Researchers can inspect evidence cards before ideation, check whether the BuildSpec matches their intended method, verify whether reported metrics are present in result files, and decide whether a failed stage should be repaired or rerun. This makes automated AI research easier to supervise, debug, and reproduce.

3.2. End-to-End Demo

We also demonstrate that QUIT can execute the full workflow end-to-end. In our demo, we instantiate QUIT on an offline RL problem:

Topic: *Flow Matching for Offline RL*
Domain: *Offline Reinforcement Learning; Policy Improvement; Flow Matching*
Objective: *Generate a literature-grounded idea, implement it, and produce a paper artifact.*

Fig. 4 shows selected pages from the generated draft.

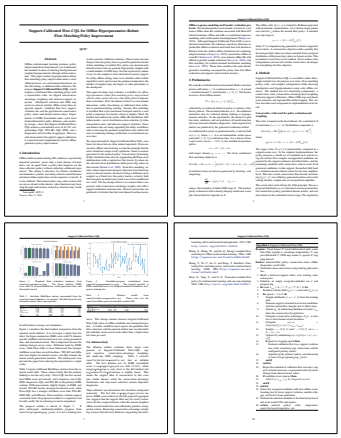

Figure 4. Selected pages from the draft generated by QUIT in the offline RL demo.

3.3. Stage-Level Demo

Beyond end-to-end execution, QUIT supports stage-level invocation. A researcher can run a single state, or specify a start state and a stop state, using artifacts already stored in the run directory.

For the same topic and domain as in Subsection 3.2, we also demonstrate partial execution by setting the initial state to `PLAN` and the stopping state to `IDEATE`. This run executes only the front part of the workflow: `PLAN` $\Rightarrow$ `VALIDATE_BRIEF` $\Rightarrow$ `RETRIEVE` $\Rightarrow$ `READ` $\Rightarrow$ `IDEATE`.

For more details on the generated artifacts, intermediate reports, and state-level outputs, please refer to Appendix B.

4. Conclusion and Limitations

We presented QUIT, a human-in-the-loop platform for AI research automation. QUIT organizes the research process into artifact-producing stages that support both end-to-end execution and reproducible partial execution. Together with the researcher-facing interface, this design turns automated AI research from a black-box pipeline into an inspectable, controllable, and resumable workflow.

Future work will explore RL-based optimization (Han et al., 2026c;a; 2025; 2026b), because this explicit state-machine design provides a natural interface.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

Anthropic. Claude opus 4.7 system card, 2026.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

Han, X., Afifi, H., and Marot, M. Elapse: Expand latent action projection space for policy optimization in offline reinforcement learning. *Neurocomputing*, 631:129665, 2025. ISSN 0925-2312. doi: https://doi.org/10.1016/j.neucom.2025.129665. URL https://www.sciencedirect.com/science/article/pii/S0925231225003376.

Han, X., Afifi, H., and Marot, M. Piql: Projective implicit q-learning with support constraint for offline reinforcement learning, 2026a. URL https://arxiv.org/abs/2501.08907.

Han, X., Afifi, H., Marot, M., Wang, X., and Yin, L. Long chain-of-thought compression via fine-grained group policy optimization. In *ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 4471–4475, 2026b. doi: 10.1109/ICASSP55912.2026.11464077.

Han, X., Fang, Q., Afifi, H., and Marot, M. Automatic constraint policy optimization based on continuous constraint interpolation framework for offline reinforcement learning, 2026c. URL https://arxiv.org/abs/2601.23010.

Jiabin Tang, Tianyu Fan, C. H. AutoAgent: A Fully-Automated and Zero-Code Framework for LLM Agents, 2025. URL https://arxiv.org/abs/2502.05957.

Liu, J., Xia, P., Han, S., Qiu, S., Zhang, L., Chen, G., Tu, H., Yang, X., Zhou, J., Zhu, H., Li, Y., Zhang, J., Zhou, Y., Zheng, Z., Xie, C., Ding, M., and Yao, H. Autoresearchclaw: Fully autonomous research from idea to paper, 2026. URL https://github.com/aiming-lab/AutoResearchClaw.

Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., McLaughlin, A., Low, A., Ostrow, A., Ananthram, A., et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.

Yang, R., Li, Y., and Li, S. Aris: Fully autonomous research via adversarial multi-agent collaboration, 2026. URL https://github.com/wanshuiyin/Auto-claude-code-research-in-sleep.

Zhang, X. Deep researcher agent: Autonomous deep learning experiment framework, 2026. URL https://github.com/Xiangyue-Zhang/auto-deep-researcher-24x7.

A. Agent Roles and Collaboration

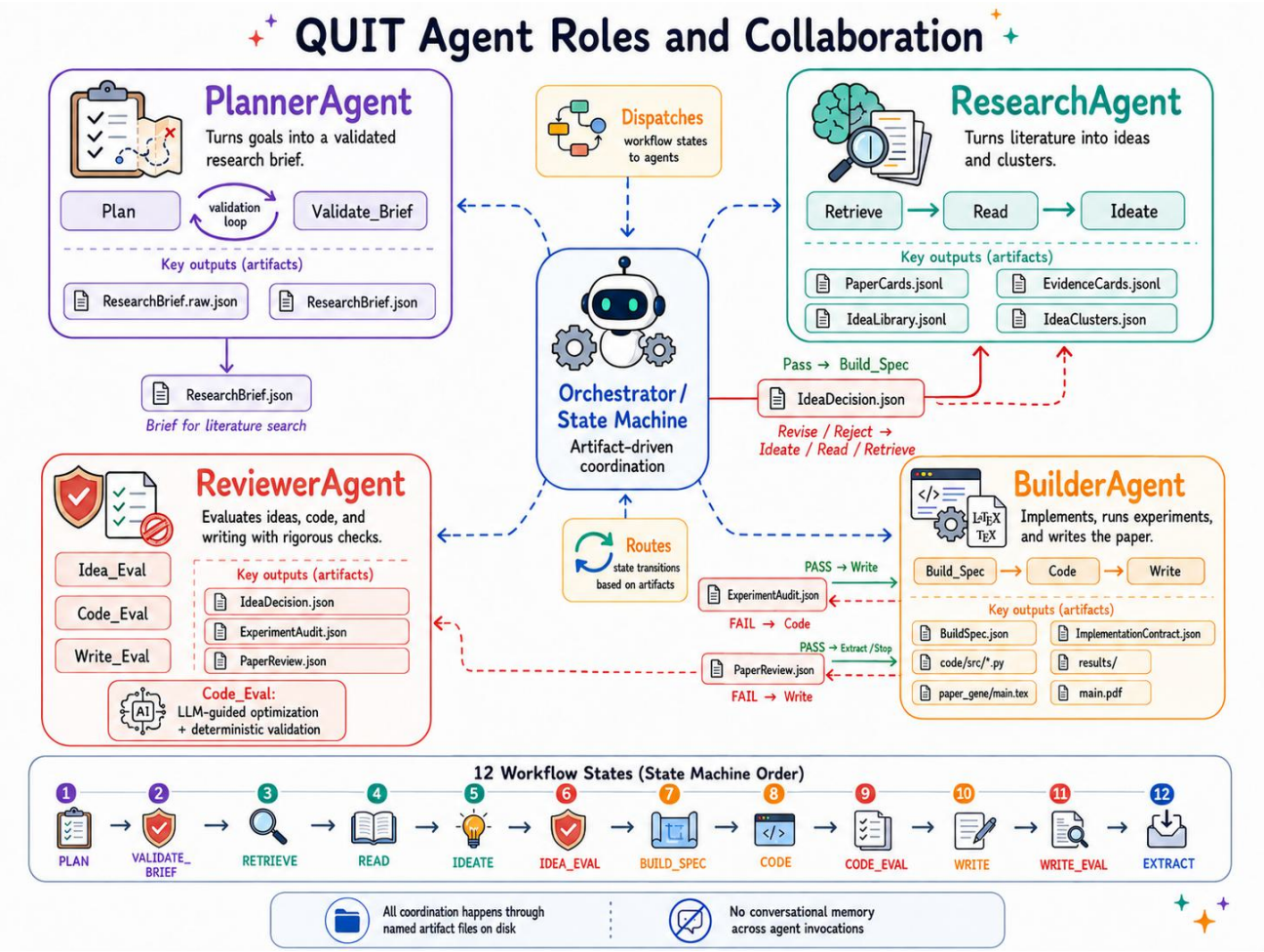

Figure 5. QUIT agent roles and collaboration. The central Orchestrator/State Machine dispatches each of the 12 workflow states to one of four specialized agents. The top-left panel corresponds to **PlannerAgent**, which handles the brief-construction loop. The top-right panel corresponds to **ResearchAgent**, which handles retrieval, reading, and ideation. The bottom-right panel corresponds to **BuilderAgent**, which handles BuildSpec generation, coding, execution, and writing. The bottom-left panel corresponds to **ReviewerAgent**, which handles idea review, LLM-guided code evaluation, and paper review. The most important interaction pattern is that review outputs do not directly change another agent's internal state; instead, they write explicit artifacts, which are then consumed on the next invocation. This makes inter-agent coordination inspectable, reproducible, and resumable. Coordination happens through explicit artifact handoffs rather than shared conversational memory.

Overview. QUIT is orchestrated by a single state machine that dispatches each pipeline stage to one of four specialized agents: **PlannerAgent**, **ResearchAgent**, **BuilderAgent**, and **ReviewerAgent**, as shown in Fig. 5. Each agent owns a bounded slice of the workflow: it reads a declared set of artifact files, performs its computation, and writes its outputs before returning control to the orchestrator. No agent maintains conversational state across invocations; all coordination happens through named artifact files on disk.

PlannerAgent is responsible for the front-end planning step. It turns the user topic into a validated `ResearchBrief.json`, which becomes the control artifact for downstream retrieval, ideation, and implementation.

ResearchAgent is responsible for literature-facing work. It retrieves papers, reads selected sources, extracts structured evidence, and synthesizes candidate ideas, producing artifacts such as `PaperCards.jsonl`, `EvidenceCards.jsonl`, and `IdeaLibrary.jsonl`.

BuilderAgent is responsible for turning a selected idea into an executable project and a paper draft. It creates

`BuildSpec.json`, generates experiment code, runs the project, collects result artifacts, and writes the manuscript.

ReviewerAgent is responsible for intermediate evaluation and failure routing. It reviews ideas, applies LLM-guided code evaluation with deterministic validation to experiment artifacts, and checks the paper draft, then writes decision artifacts that determine whether the workflow advances or loops back for repair.

Collaboration through artifact handoffs. Fig. 5 illustrates how the four agents collaborate through the central Orchestrator/State Machine. The orchestrator dispatches each workflow state to the appropriate agent, receives the result, writes the output artifact to disk, and routes to the next state based on a deterministic transition rule. No agent calls another agent directly; as the figure caption notes, *"all coordination happens through named artifact files on disk"* and there is *"no conversational memory across agent invocations."* The bottom row of the figure shows the 12 workflow states in state-machine order, while the four panels above summarize the responsibilities of the four agents.

B. Web Interface: Stage Invocation and Reproducibility

Overview. QUIT ships with a browser-based monitoring and control panel that exposes the full pipeline over HTTP without requiring any command-line interaction after the server is started. Fig. 6 shows the interface in its idle state. The panel is divided into three regions: a *pipeline bar* across the top for selecting which stages to execute, a *configuration panel* on the left for editing the research topic and LLM credentials, and an *output area* on the right for launching runs and streaming live logs.

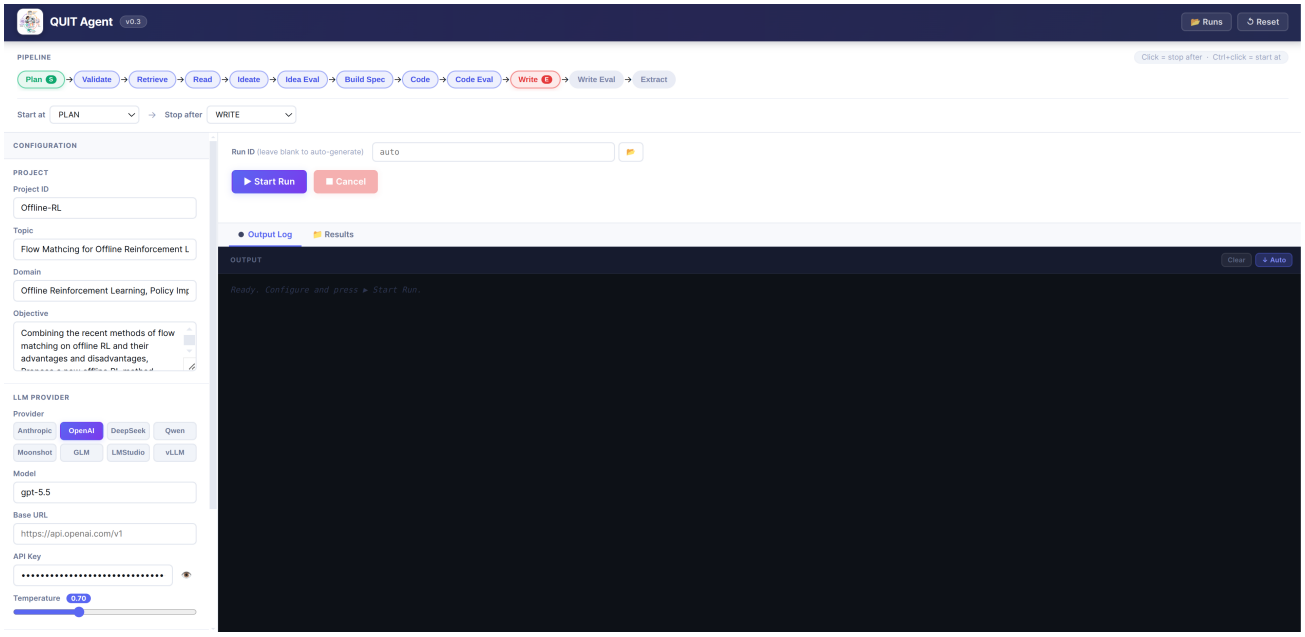

Figure 6. QUIT web interface in idle state. The pipeline bar (top) shows all twelve workflow stages; the green chip (PLAN, badge S) marks the current start-at state and the red chip (WRITE, badge E) marks the stop-after state. The configuration panel (left) exposes LLM provider, model, and research topic settings. The output area (right) contains the run-ID field and the **Start Run** button.

Pipeline bar and stage selection. The pipeline bar (Figure 7) is the primary mechanism for controlling which stages execute. It renders all twelve `WorkflowState` values as a horizontal chain of clickable chips: PLAN → VALIDATE → RETRIEVE → READ → IDEATE → IDEA EVAL → BUILD SPEC → CODE → CODE EVAL → WRITE → WRITE EVAL → EXTRACT.

- **Single click** on a chip sets the *stop-after* state: the pipeline will halt after that stage completes. The hint line above the bar reads *"Click = stop after"*.
- **Ctrl+click** on a chip sets the *start-at* state: the pipeline will enter the state machine at that stage, loading artifacts produced by earlier stages from the run directory on disk. The hint line reads *"Ctrl+click = start at"*.

The selected range is highlighted between the two chosen chips. The *Start at* and *Stop after* dropdown selectors below the bar stay synchronized with the chip selection and can also be used directly. A badge `S` marks the current start-at chip in green; a badge `E` marks the stop-after chip in red.

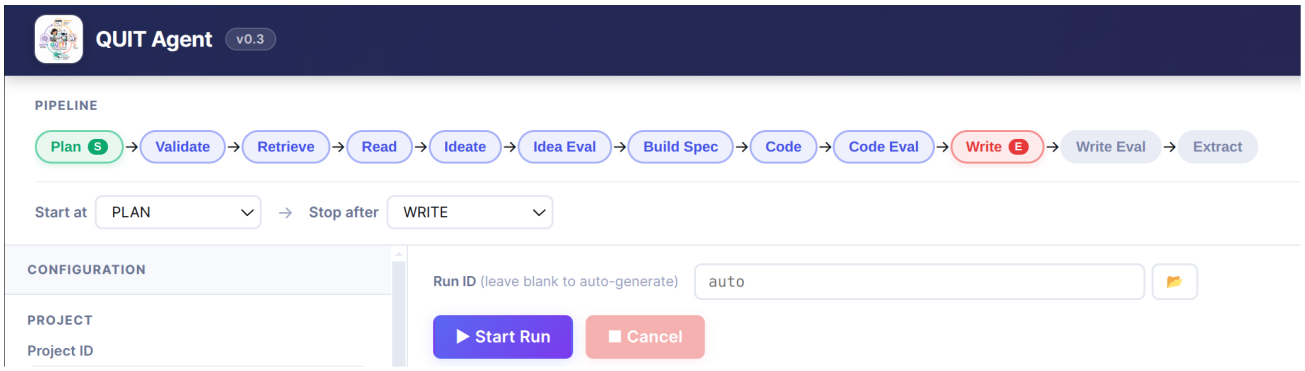

Figure 7. Pipeline bar and run launcher (detail). The PLAN chip (green, `S`) is the start-at state; the WRITE chip (red, `E`) is the stop-after state, matching the *Start at / Stop after* dropdowns below. The **Run ID** field accepts an existing run directory name for resume invocations; leaving it blank auto-generates a new ID. **Start Run** serializes the configuration form and posts to `POST /api/run`; **Cancel** sends `POST /api/jobs/<id>/cancel` to terminate the subprocess.

Stage-range invocation: three representative scenarios. All partial-execution patterns are driven by the same two gestures—single click (stop-after) and Ctrl+click (start-at)—followed by pressing **Start Run**.

*Scenario A: Re-run only ideation and idea selection (*IDEATE→IDEA_EVAL*).* Click the **Runs** button (top right) to open the run history modal and select the target run; this populates the Run ID field. Ctrl+click the IDEATE chip to set it as the start-at state. Single-click the IDEA EVAL chip to set it as the stop-after state. The highlighted range now covers exactly these two chips. Press **Start Run**. The live log pane streams progress; on completion, switch to the **Results** tab and click `IdeaDecision.json` in the file tree to confirm `"decision": "PASS"`.

*Scenario B: Regenerate the experiment contract and code without re-reading papers (*BUILD_SPEC→CODE_EVAL*).* With the same run ID selected, Ctrl+click BUILD SPEC and single-click CODE EVAL. Press **Start Run**. When the run finishes, expand the `results/` directory in the file tree and click `ExperimentAudit.json` to verify `"status": "PASS"` and an empty `failures` list. Clicking `results/metrics.json` or `results_table.csv` previews numeric outputs inline without leaving the browser.

*Scenario C: Revise the paper draft only, using existing result artifacts (*WRITE→WRITE_EVAL*).* Ctrl+click WRITE and single-click WRITE EVAL. Press **Start Run**. On completion, click `paper_gene/main.pdf` in the file tree; the browser's native PDF viewer renders the compiled paper inline. Click `PaperReview.json` to confirm `"status": "PASS"` and inspect any remaining revision notes.

Expected outputs. Tab. 1 lists the key files produced by each scenario and the verification signal accessible via the file preview panel. All paths are relative to `runs/<run-id>/`.

Verifying workflow success via the Results tab. Switching to the **Results** tab after selecting a run ID loads a recursive file tree of the run directory. Seven key artifact files are visually highlighted in the tree (`BuildSpec.json`, `ResearchBrief.json`, `IdeaDecision.json`, `ExperimentAudit.json`, `paper_gene/main.pdf`, `EXPERIMENT_LOG.md`, `CodeRunReport.json`), and the top-level directories `paper_gene/`, `code/`, and `results/` are auto-expanded on load. Clicking any file opens it in the preview panel: JSON files are pretty-printed, PDFs are rendered inline, and other text files (`.tex`, `.py`, `.md`, `.jsonl`, `.csv`) appear as plain text. The primary reproducibility checks are:

- `ExperimentAudit.json` — deterministic validation verdict recorded for CODE_EVAL, with a `failures` list naming every unsatisfied validation rule;
- `PaperReview.json` — pass/fail verdict for WRITE_EVAL, with a `failures` list for each unsatisfied writing check;

Table 1. Expected outputs and browser-accessible verification signals for end-to-end and stage-level QUIT executions. All paths are relative to `runs/<run-id>/` and can be opened directly in the Results tab file preview.

Stage range	Key output files	Verification signal
End-to-end (PLAN→EXTRACT)	`ResearchBrief.json`, `BuildSpec.json`, `results/metrics.json`, `results/results_table.csv`, `paper_gene/main.pdf`, `PaperReview.json`, `run_trace.json`	`PaperReview.json`: `status = "PASS"`; `run_trace.json`: last entry `state = "EXTRACT"`, `status = "PASS"`
IDEATE→ IDEA_EVAL	`IdeaLibrary.jsonl`, `IdeaClusters.json`, `IdeaDecision.json`	`IdeaDecision.json`: `decision = "PASS"`; `IdeaLibrary.jsonl`: at least one record with `supporting_evidence_ids` non-empty
BUILD_SPEC→ CODE_EVAL	`BuildSpec.json`, `code/src/*.py`, `results/metrics.json`, `results/results_table.csv`, `results/*.png`, `ExperimentAudit.json`, `EXPERIMENT_LOG.md`	`ExperimentAudit.json`: `status = "PASS"`; `CodeRunReport.json`: `returncode = 0`; `results/metrics.json` parseable as JSON object
WRITE→ WRITE_EVAL	`paper_gene/main.tex`, `paper_gene/references.bib`, `paper_gene/main.pdf`, `WriteReport.json`, `PaperReview.json`	`PaperReview.json`: `status = "PASS"`; `WriteReport.json`: `page_validation.status = "PASS"`; `paper_gene/main.pdf` exists and has $\geq$8 pages

- `run_trace.json` — ordered list of all executed steps with `state`, `next_state`, `status`, and `timestamp`;
- `llm/<step>_prompt.txt` and `llm/<step>_response.txt` — verbatim LLM prompt/response pairs for every generation decision, enabling offline inspection without re-querying the provider.

A run whose `PaperReview.json` records `status = "PASS"` and whose `ExperimentAudit.json` records `status = "PASS"` has passed the recorded validation gates.

Implementation status and limitations. The interface is a functional prototype, not a polished research product. Current limitations include: (i) file *viewing* only—artifact files must be edited on the filesystem; (ii) no retrospective per-stage color coding on the pipeline bar after a run completes—the researcher must open `ExperimentAudit.json` and `PaperReview.json` manually; (iii) job state is held in server memory and lost on restart; (iv) no authentication, intended for local or trusted-network use only; (v) the EXTRACT stage output is not yet highlighted in the file tree. Future work includes inline artifact editing, persistent job history, and a per-stage status dashboard that reads audit reports automatically.

C. Artifact Schema and Run Directory

Motivation. A central design principle of QUIT is that *all intermediate research state is externalized into durable, named files* rather than retained implicitly in agent conversation history. Each workflow stage reads a declared set of input artifacts, performs its computation, and writes a declared set of output artifacts before handing control to the next stage. This separation has three practical consequences. First, any artifact can be inspected or edited by a human between stages, enabling transparent human-in-the-loop intervention. Second, a failed or interrupted run can be resumed from any checkpoint by supplying the artifacts already produced and re-entering the state machine at the appropriate stage. Third, the artifact boundary makes each stage independently auditable: a downstream stage can verify that required inputs satisfy schema constraints before proceeding, rather than relying on opaque conversational context.

Run directory structure. Every execution of QUIT creates a timestamped run directory under `runs/<run-id>/`. The following tree shows the complete layout for the representative Offline-RL run (`runs/Offline-RL - 20260510081500/`).

```
runs/Offline-RL - 20260510081500/
|-- run_trace.json                    # ordered step summary
|-- trace/                            # per-step execution traces
```

```
|   |-- 0001_plan.json
|   |-- 0002_validate_brief.json
|   |-- 0003_retrieve.json
|   |-- 0004_read.json
|   |-- 0005_ideate.json
|   |-- 0006_idea_eval.json
|   |-- 0007_build_spec.json
|   |-- 0008_code.json
|   |-- 0009_code_eval.json
|   |-- 0010_write.json
|   |-- 0011_write_eval.json
|   `-- 0012_extract.json
|-- llm/                              # raw prompt / response pairs
|   |-- 0001_plan_prompt.txt
|   |-- 0001_plan_response.txt
|   `-- ...  (one pair per stage)
|
|-- ResearchBrief.raw.json            # raw LLM output from PLAN
|-- ResearchBrief.json                # validated brief (human-editable)
|-- ResearchBriefValidationReport.json
|-- RetrievalReport.json
|-- PaperCards.jsonl                  # retrieved paper metadata
|-- paper_retrieve/                   # downloaded PDFs
|   `-- <arxiv-id>.pdf
|-- PaperTexts.jsonl                  # extracted full text
|-- EvidenceCards.jsonl               # structured evidence records
|-- EvidenceValidationReport.json
|-- ReadReport.json
|-- RepoCards.jsonl                   # repository links from papers
|-- repos/nju_rl_meta_dt/             # cloned reference repository
|-- RepoCloneReport.json
|-- RepoRetrievalReport.json
|
|-- IdeaClusters.json                 # evidence clusters
|-- IdeaLibrary.jsonl                 # candidate ideas (human-editable)
|-- IdeaGenerationReport.json
|-- IdeaDecision.json                 # selected idea + fallback routing
|
|-- BuildSpec.raw.txt                 # raw LLM output from BUILD_SPEC
|-- BuildSpec.json                    # central experiment contract
|-- ImplementationContract.json
|-- RepoAdaptationReport.json
|
|-- code/                             # generated experiment project
|   |-- src/
|   |   |-- dataset.py
|   |   |-- method.py
|   |   |-- baselines.py
|   |   |-- train.py
|   |   |-- evaluate.py
|   |   `-- plot.py
|   |-- run_experiment.py
|   `-- configs/experiment_config.json
|
|-- EnvironmentResolutionReport.json
|-- CodeSyntaxReport.json
|-- DependencyInstallReport.json
|-- DatasetAcquisitionReport.json
|-- DeviceReport.json
|-- RuntimePatchReport.json
|-- CodeRunReport.json                # execution status (return code, errors)
|-- CodeRepairReport.json
|-- EXPERIMENT_LOG.md
|-- CodeEvalQualityReport.json
|-- ExperimentAudit.json              # CODE_EVAL validation result
```

```
|-- CodePerformanceEval.json          # LLM performance review
|-- CodeReviseReport.json             # code revision loop report
|-- ResultCollectionReport.json
|-- PaperBaselineResults.json
|
|-- results/                          # produced by code execution
|   |-- metrics.json
|   |-- results_table.csv
|   |-- summary_table.csv
|   |-- per_seed_results.csv
|   |-- ablation_results.csv
|   |-- sensitivity_results.csv
|   |-- progress_log.jsonl
|   |-- progress_curve.png
|   `-- eval_curve.png
|
|-- paper_gene/                       # generated paper
|   |-- main.tex
|   |-- references.bib
|   `-- main.pdf
|
|-- PaperReview.json                  # WRITE_EVAL review result
`-- extract/                          # terminal EXTRACT outputs
    |-- research_notes.md
    `-- skill_candidates.json
```

Artifact registry. Tab. 2 lists the major artifacts, the stage that produces each one, the downstream stages that consume it, and the artifact's role in the pipeline. Artifacts marked *human-editable* have self-contained JSON or plain-text schemas; a researcher can modify them between stages and the system will use the modified version on the next invocation.

Table 2. Major QUIT artifacts: production, consumption, and purpose. † denotes artifacts that are human-editable between stages. ‡ denotes artifacts used for validation, audit, or failure recovery.

Artifact	Produced by	Consumed by	Purpose
`ResearchBrief.json`†	PLAN, VALIDATE_BRIEF	RETRIEVE, READ, IDEATE	Defines topic, scope, constraints, search budget
`ResearchBriefValidationReport.json`‡	VALIDATE_BRIEF	PLAN (on retry)	Schema errors used to guide re-planning
`PaperCards.jsonl`	RETRIEVE	READ	Paper metadata (title, authors, abstract, URL)
`paper_retrieve/*.pdf`	RETRIEVE	READ	Downloaded full-text PDFs for evidence extraction
`EvidenceCards.jsonl`†	READ	IDEATE, BUILD_SPEC, WRITE	Structured claims, metrics, transferable seeds per paper
`RepoCards.jsonl`	RETRIEVE	BUILD_SPEC, CODE	Repository URLs and metadata from retrieved papers
`repos/<name>/`	BUILD_SPEC	CODE	Cloned reference codebase (read-only context)
`IdeaLibrary.jsonl`†	IDEATE	IDEA_EVAL, BUILD_SPEC	Candidate research ideas with evidence links and scores
`IdeaDecision.json`†	IDEA_EVAL	BUILD_SPEC	Selected idea ID, PASS/REVISE/REJECT verdict, fallback target
`BuildSpec.json`†	BUILD_SPEC	CODE, CODE_EVAL, WRITE, WRITE_EVAL	Central contract: method, metrics, baselines, environment
`ImplementationContract.json`	CODE	CODE (sub-stages)	API surface (class names, method signatures) for code generation
`code/src/*.py`	CODE	CODE (execution)	Generated experiment modules (dataset, method, baselines, ...)
`CodeRunReport.json`‡	CODE	CODE_EVAL	Return code, stdout/stderr summary, execution metadata
`ExperimentAudit.json`‡	CODE_EVAL	CODE (repair), WRITE	Validation verdict and list of failures
`CodePerformanceEval.json`‡	CODE_EVAL	CODE (revision), WRITE	LLM-guided performance verdict and revision hints
`CodeReviseReport.json`‡	CODE (revision)	CODE_EVAL	Summary of code changes made after performance feedback
`EXPERIMENT_LOG.md`‡	CODE	CODE_EVAL, WRITE	Human-readable execution log (build ID, return code, status)
`results/metrics.json`‡	CODE	CODE_EVAL, WRITE	Key scalar metrics consumed by audit and paper tables
`results/results_table.csv`	CODE	CODE_EVAL, WRITE	Per-method, per-seed result rows for all BuildSpec metrics
`results/summary_table.csv`	CODE	WRITE	Compact paper-facing result rows (aggregated)
`results/*.csv` (ablation, sensitivity, per-seed)	CODE	WRITE	Secondary experiment tables for appendix
`results/*.png` (progress_curve, eval_curve)	CODE	WRITE, WRITE_EVAL	Figures embedded in paper; Write_Eval checks inclusion
`results/progress_log.jsonl`	CODE	WRITE	Epoch-level training log; used to generate progress curves
`paper_gene/main.tex`	WRITE	WRITE_EVAL	Raw LaTeX source (no markdown wrapper)
`paper_gene/references.bib`	WRITE	WRITE (compile), WRITE_EVAL	Bibliography; keys must match BuildSpec citation list
`WriteReport.json`‡	WRITE	WRITE_EVAL, WRITE (revision)	Compilation status, page count, version history
`PaperReview.json`‡	WRITE_EVAL	WRITE (revision)	Review verdict; failures trigger targeted revision
`trace/<step>.json`‡	Every stage	Post-hoc inspection, resumption	Input/output list, prompt path, validation result, next-state routing
`run_trace.json`‡	Every stage	Post-hoc inspection, resumption	Ordered summary of all executed steps